

Software Architecture for Multi-User Multiplexing to Enhance Scalability in IoT-Based Remote Labs

Akshit Gureja, Rishabh Agrawal, Sachin Chaudhari, Karthik Vaidhyanathan, Venkatesh Choppella
International Institute of Information Technology-Hyderabad (IIIT-H), India
Emails: akshit.gureja@research.iiit.ac.in, rishabh.agrawal@students.iiit.ac.in,
{sachin.chaudhari, karthik.vaidhyanathan, venkatesh.choppella}@iiit.ac.in

Abstract— Remote Labs refer to an end-to-end system, including hardware and software built to access scientific equipment and resources remotely. The software platform built for such purposes needs to be robust enough to handle the communication of inputs and outputs between the client and the hardware nodes with minimal latency and simultaneously provide a seamless user experience. This paper highlights the importance of scalability in Remote Labs and presents multi-user multiplexing as a solution, to essentially provide users with concurrent access to the hardware node for experiments which can generate outputs instantaneously. The paper discusses the inefficiency of existing web-based Remote Labs with 4-layered architectures and proposes the use of WebSocket with a 3-layer software architecture to enhance user experience, accelerate input-output communication and implement multi-user multiplexing. To showcase the effectiveness of the proposed architecture over existing implementations using Blynk IoT platform as the middleware, a comprehensive communication pipeline was developed from scratch to perform Kirchhoff's Voltage Law (KVL) experiment remotely.

Index Terms—Blynk, Internet of Things (IoT), Remote Labs, Scalability, Software Multiplexing, WebSockets

I. INTRODUCTION

The Internet of Things (IoT) has constantly revolutionized various sectors and penetrated our daily activities. It has transformed lives by enabling seamless communication and data exchange in healthcare, transportation, smart homes, agriculture and logistics [1]. Remote Labs, an IoT application in the education sector, aims to provide users with access to scientific laboratories, expensive and delicate scientific equipment and perform experiments using an online platform [2]–[5]. Using Remote Labs, students can conduct experiments remotely, get real-time output data and analyze results interactively, thus helping to enhance flexibility and convenience by supporting distance and self-paced learning in times of limited physical access. Moreover, Remote Labs can bridge the gap in learning and be highly beneficial for institutions suffering from poor laboratory infrastructure. They were extremely valuable during the Covid-19 pandemic as the problem of limited access extravagated due to lockdowns.

Implementing a Remote Lab system typically entails developing an architecture that intricately synchronizes the hardware and software pipelines specifically designed to facilitate remote experimentation and handle the communication between a client and the experiment's hardware. Multiple

universities and institutions around the world have created and developed Remote Labs previously, including UNILabs [6] and iLabs at MIT [7]. Remote Labs were also built at IIIT-Hyderabad to conduct the Conservation of Mechanical Energy [8] and Vanishing Rod experiments [9]. The European Go-Lab initiative [10], LabsLand Network [11], LABERSIME cloud platform [12] are among multiple other implementations.

With an ever-increasing school-going young population, Remote Labs are required to handle large loads motivating such systems to improve their scalability and increase concurrent access. For such purposes, methods need to be developed to manage the potential strain that might come on systems when a significant number of students attempt to access a specific experiment or the platform as a whole. There exist Remote Lab implementations such as [9], which earlier tried to scale the usability of experiments involving mechanical actuation by miniaturizing hardware setups and employing computer vision techniques on a single camera. Others have improved scalability by developing cost-effective architectures [13].

Remote Lab experiments involving mechanical movements present a unique challenge for scaling due to requirement of single-user access, as multiple students cannot simultaneously control motor actuation. For such experiments, several methods are available to manage the supply and demand pipeline effectively. Some of these are hardware multiplexing, queues and slot bookings. Hardware multiplexing involves horizontal scaling by increasing the number of hardware nodes dedicated to an experiment to cope with high demand. Users are hence redirected to different nodes based on their occupancy and availability. Queues are an improvement on the demand side by enforcing single access and ensuring a First-Come, First-Server (FCFS) or First-In, First-Out (FIFO) scheduling approach among users. Slot booking is again a demand-side management feature as it distributes demand for accessing experiments across predetermined time intervals which can be reserved for exclusive access and alleviate high loads.

However, another domain of experiments involving rapid settling of electric currents can provide near-instantaneous input-output responses, and this characteristic feature can be leveraged for scaling Remote Lab implementations. Although there is still single access to the hardware, the communication channel now allows multiple users to conduct experiments almost simultaneously without impacting any other user's experience. This feature of the architecture can be referred

to as multi-user multiplexing and is the focus of this paper.

The specific contributions presented through this paper are:

- The concept of multi-user multiplexing along with its demonstration in experiments which provide instantaneous outputs, such as the KVL experiment.
- Implementation of general 4-layer architecture (Fig: 1) for web-based Remote Labs using Blynk as middleware (similar to [9]) for data transmissions and show its incapability in supporting multi-user multiplexing.
- A 3-layer architecture leveraging WebSockets without middleware has been proposed to implement multi-user multiplexing and provide a solution to scaling Remote Labs by satisfying simultaneous user access.
- A comparison of performance and efficiency between the existing 4-layered implementation using Blynk and the proposed 3-layered architecture.

None of the papers, to the best of our knowledge, have tried the approach of multi-user multiplexing to scale Remote Lab implementations by providing near-simultaneous user access.

The subsequent sections of the paper are as follows: Section II provides an overview of the experiment and hardware components used. Section III discusses an ideal multiplexing scenario, outlines the requirements of a Remote Lab system to allow multiple user access. Section IV proposes a 3-layer architecture that uses WebSockets for multi-user multiplexing. Section V discusses the criterion used for comparison and explains the results obtained based on conducted tests.

II. REMOTE LAB FOR KIRCHHOFF'S VOLTAGE LAW EXPERIMENT

A. Theory

KVL is one of the fundamental principles in electrical circuit analysis. It is named after the German physicist Gustav Kirchhoff and is also known as Kirchhoff's first law. It is based on the principle of conservation of energy.

Theorem: Sum of all voltages around any closed loop in an electrical circuit is equal to zero. [14]

In other words, in a closed loop of a circuit, the sum of the voltage rises and drops must be zero. Voltage rise occurs across voltage sources (like batteries) and other active circuit elements, while voltage drops occur across resistors and other passive elements. The equation for this can be written as,

$$\sum V_{(\text{of all sources})} - \sum V_{(\text{across passive elements})} = 0 \quad (1)$$

For example, in Fig. 2, there are three closed loops. The first loop contains a battery, R_2 , R_3 , and R_4 , the second loop has R_1 , R_4 , battery and the last loop has R_1 , R_2 and R_3 . KVL equations for these loops can be written as:

$$\text{Loop1: } 3.3 - V_{AB} - V_{BC} - V_{CD} = 0 \quad (2)$$

$$\text{Loop2: } 3.3 - V_{AC} - V_{CD} = 0 \quad (3)$$

$$\text{Loop3: } V_{AC} + V_{CB} + V_{BA} = 0 \quad (4)$$

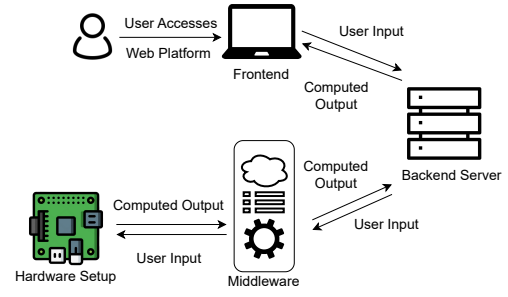


Fig. 1: A general architecture for web based Remote Labs with frontend, backend server, middleware and hardware node.

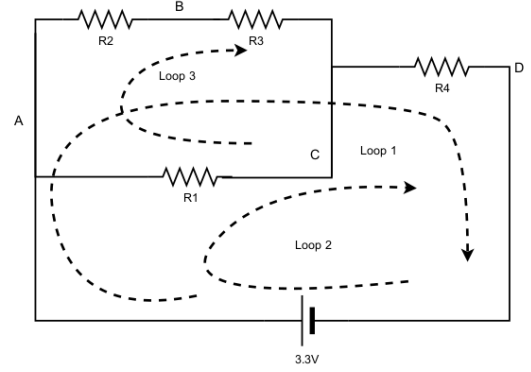


Fig. 2: Example to illustrate KVL experiment.

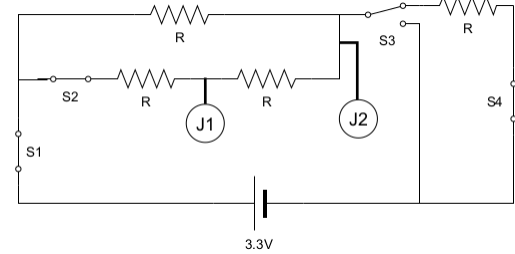
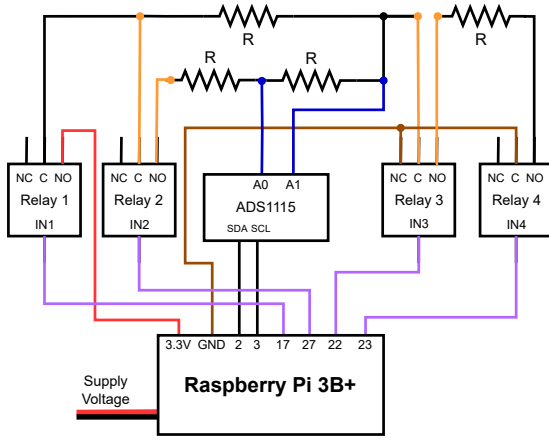


Fig. 3: Circuit implemented to verify KVL with relays to use the circuit in different configurations and measure the voltage at junctions J_1 , J_2 .

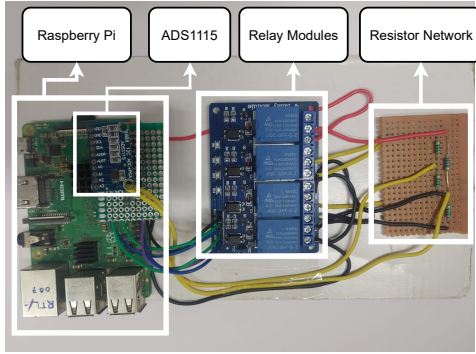
B. Hardware for Deployed Remote Lab

For the verification of KVL, a setup of a simple circuit of resistors has been made, including, a power source and an analog-to-digital converter (ADC) which has been used as a voltmeter to measure the voltage across different components of the circuit and then take the algebraic sum of these voltages which should ideally be equal to zero.

The circuit depicted in Fig. 3 has been implemented in hardware for the verification of the KVL experiment with identical resistor values of 100Ω each (depicted as R in further diagrams). The hardware consists of relays which are controlled using a Raspberry Pi 3B+. These relays can act as switches and provide multiple configurations from the same electric circuit, as shown in Fig. 3, along with the ability to toggle between them. For measuring voltages at junctions, an external ADC (ADS1115) is used as a voltmeter. The circuit diagram for the hardware is in Fig. 4a while the deployed Remote Lab for the KVL experiment is in Fig. 4b.



(a) Circuit diagram of KVL experiment.



(b) Hardware for KVL experiment.

Fig. 4: Deployed Remote Labs for KVL experiment.

Switching relays provides the user with the ability to use the hardware in different circuit configurations. There are four configurations possible for the hardware deployed. The first configuration is a single resistor connected to a power source, the second configuration makes a series combination with two resistors. The third and fourth gives a combination of resistors in parallel and series with each other. Circuit diagrams of all the configurations are provided in Fig. 5.

Electromagnetic relays are used to toggle between these configurations. Each toggle operation takes about 10-15 ms and settles to stable values at all the junctions. After this delay, the external ADC measures voltages at two different junctions in the circuit, shown by blue wires in Fig. 4a.

Table. I shows all combinations of relay states for changing into different modes of the circuit. S_1 , S_2 , S_3 , and S_4 are the four relay switches. As shown in Fig. 3, J_1 and J_2 are junctions where values are calculated. For 4th configuration, the circuit is a combination of series and parallel circuits. The four relays toggle to set the circuit in the desired combinations. As given in the table. I, for the fourth combination, all relays are in the active stage. Then we have two junctions, J_1 and J_2 , where voltage is measured Fig. 3 using ADS1115 ADC.

C. Software using Existing Architecture

Following the Remote Labs implementation using Blynk as described in [9], an end-to-end Remote Labs system has been developed as depicted in Fig. 6. The system replicates

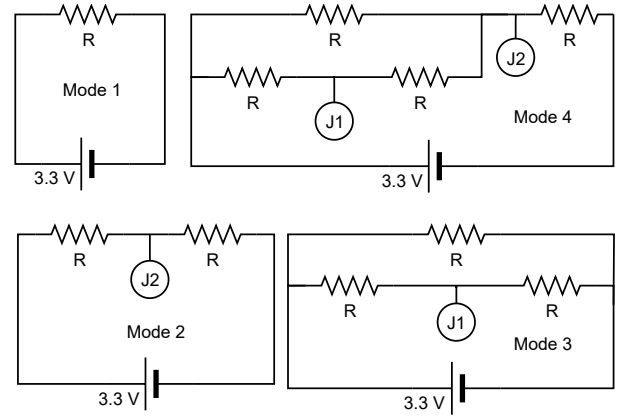


Fig. 5: Four modes of circuit operation.

TABLE I: Relay state and voltages for all the combinations.

Mode	S1	S2	S3	S4	J1	J2
1	On	Off	Off	Off	X	0
2	On	Off	On	On	X	1.65
3	On	On	Off	Off	1.65	0
4	On	On	On	On	2.64	1.98

the 4-layer architecture using Blynk Cloud as the middleware. The system comprises of 4-layers as claimed - a frontend, the backend server, the Blynk Cloud as middleware and the Raspberry Pi. The frontend allows the user to choose their desired mode of circuit operation as an input with a Node.js server that is used to make the platform robust, efficiently handle the incoming client requests and communicate with the Blynk Cloud connected to the hardware. Fig. 6 shows the interaction of the user with the platform using dotted arrows, whereas the six steps involved in the communication pipeline are depicted by solid arrows and are Roman numbered.

The developed platform employs Axios, a promise-based HTTP client library [15], which handles Step I of sending the input values from the frontend to a Node.js server whenever the user selects one of the four modes of circuit operation. When an Axios call is made, a promise is returned to the user which is resolved only upon receiving the output. Blynk in its developer mode enables clients to send, modify and receive the values on the Blynk Cloud using Representational State Transfer (REST) Application Programming Interfaces (APIs) [16]. Step II involves the server updating the virtual pins on the Blynk Cloud using API calls upon receiving the input. Step III is Blynk sending the input of selected mode of operation to the hardware, which computes the output and updates them back to the cloud as part of Step IV. Step V is the server using GET requests to retrieve the values from the cloud which are returned to the user in Step VI by resolving the previously returned promise.

III. MULTI-USER MULTIPLEXING FOR REMOTE LAB SCALABILITY

A. Requirements of Multi-User Remote Lab Systems

Fig. 7 shows the general architecture of a system to provide the functionality of multi-user multiplexing and almost simul-

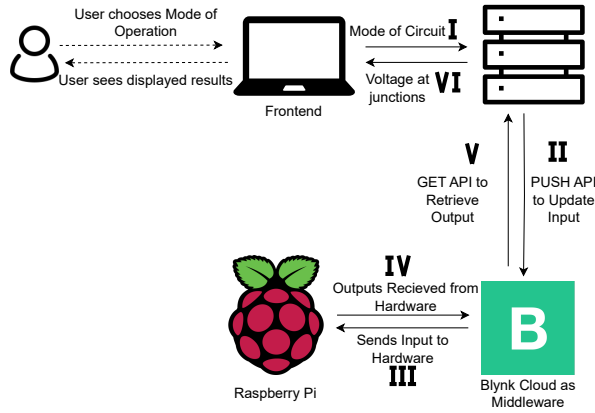


Fig. 6: Remote Labs implementation for KVL using 4-layer architecture with Blynk middleware, having a six-step round trip communication.

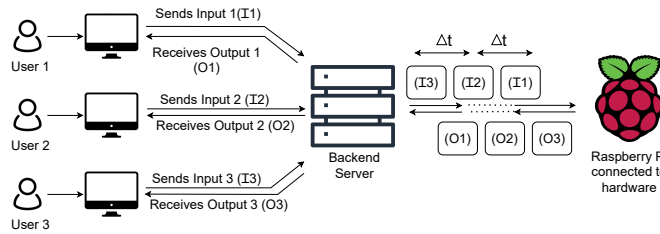


Fig. 7: Four primary requirements for simultaneous user access - single channel for server to communicate with hardware, queuing of obtained inputs for sequential and FCFS computation of outputs which are then mapped to respective clients.

taneous user access, for which it must essentially satisfy four primary requirements given below:

- 1) *Single Channel Access* - As shown in Fig. 7, the server and Raspberry Pi would have a single predecided dedicated channel on which they would communicate via some predetermined communication method.
- 2) *Message Ordering* - As the communication happens over a single channel and clients give inputs simultaneously, it is necessary to order the message in a queue as the nodes can process only one value at a time due to the single-threaded nature of hardware devices. It also ensures that an FCFS execution of inputs is maintained. Hence, as shown in Fig. 7, if input I_2 is queued after I_1 , the corresponding output O_2 will be obtained after O_1 .
- 3) *Server Sent Events* - As the inputs are received on the server and transmitted to an already connected Raspberry Pi for experimentation, this corresponds to the utilization of server-sent events (SSE). Raspberry Pi being the client, waits for inputs and SSE would enable asynchronous, unidirectional updates of input from the server to Raspberry Pi, which is the client here. This is different from the request-response architecture.
- 4) *Mapping outputs to inputs*: As there is a solitary node and server serving multiple users, it is important that the architecture should enable the software to appropriately associate the sequentially acquired output with the cor-

responding input values and accurately transmit them to the client. It is crucial that the clients exclusively receive the outputs corresponding to their specific inputs. As illustrated in Fig. 7, user 1, giving input I_1 should obtain the output O_1 computed corresponding to I_1 .

These requirements are the prerequisites for a software architecture to satisfy simultaneous user access.

B. Ideal Multi-user Multiplexing

Multi-user multiplexing in context of Remote Labs can be referred to as the ability of system to allow multiple users to access a hardware node and simultaneously perform an experiment. A multi-user multiplexing system built for electric current-based experiments should ideally provide students with instantaneous results and outputs with no latency involved because of the small response time of resistors, which gives rapidly settling electric currents and voltages in the KVL experiment. Hence, considering no network latency or output computation time, results are available in electric circuits instantaneously and this characteristic property is what allows multi-user multiplexing to be applied in such experiments.

Hence, theoretically, inputs can be given at a time interval Δt with $\Delta t \rightarrow 0$ (Fig. 7), queued at the server and the outputs can also be received sequentially, allowing thousands of users to access and experiment simultaneously. Such systems should hence be able to handle large loads efficiently without adding any latency to any user's experience, even when thousands of users are performing the experiment concurrently and communicating to the same hardware node of the KVL experiment.

Moreover, according to research in Human-Computer Interaction [17], a delay of more than 0.1 seconds is noticeable to humans indicating that the system does not react instantaneously. A delay of more than one second is considered high and is believed to interrupt the user's flow of thought. Hence, the aim is to provide outputs to the user at a latency less than the perceivable limit of one second, which can be used to implement multi-user multiplexing.

C. Practical Constraints, Challenges and Limitations of Existing Implementations

Due to practical limitations, such as the hardware's single-threaded nature for output computation, and latency in communication, there is bound to be a time interval between giving an input and receiving the corresponding output. These practical constraints restrict the number of users that are able to perform the experiment simultaneously and get accurate outputs.

The Remote Lab system in [9] utilizes Blynk Cloud and its proprietary protocol [18] for handling SSEs and transmitting updated input variable values to the hardware. Blynk is the sole entity communicating with hardware ensuring single-channel access. However, this implementation lacks input-output mapping and message ordering. Even with the addition of Node.js server for KVL experiment, it remains susceptible to incorrect mapping due to shared cloud variables. Therefore, a new architecture is needed to provide near-simultaneous user access and leverages the human eye's inability to differentiate

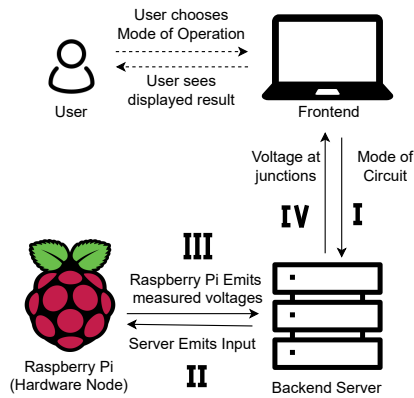


Fig. 8: 3-Layer (frontend, backend, hardware node) webSocket architecture with four-step round trip communication.

between an ideal scenario and a non-ideal scenario with non-perceivable latency, achieving real-time experimentation.

IV. PROPOSED 3-LAYER SOFTWARE ARCHITECTURE FOR SIMULTANEOUS ACCESS

This paper proposes using WebSockets in a 3-layer architecture for Remote Lab applications to communicate input and output values directly between backend server and hardware node as depicted in Fig. 8. This is in contrast to the generic 4-layer architectures using Blynk or similar middleware.

A. Key Design Decisions

The Remote Lab implementation using Blynk was incapable of message ordering and input-output mapping. To add these functionalities, the incorporation of a central backend server using Node.js is proposed to exploit its single-threaded, event-driven and asynchronous nature to handle concurrency with a built-in event loop and call-stack. This ensures that the responses obtained are mapped inherently by the server to the correct clients and that messages are ordered in a FIFO-based queue and executed on an FCFS basis.

Moreover, previous implementations having a 4-layer architecture are prone to inconsistent, unsynchronized and ill-timed server access to middleware such as Blynk Cloud in times of simultaneous access which might lead to mismatch of inputs to outputs at cloud itself, even after using a Node.js server because of uneven, and unsymmetrical updation of variables which is very delicate and difficult to manage and synchronize. Hence, highlighting possible enhancement of the system at improving functionality by removing the necessity of cloud middleware from implementation. This converts our traditional 4-layer architecture to proposed 3-layer architecture.

To handle server-sent events of input updation at the hardware without middleware and also receive outputs, the usage of WebSockets is proposed. Although SSEs are used for unidirectional asynchronous updates, a bidirectional protocol such as WebSockets can provide an alternative to SSEs by sending inputs and receiving outputs back at the server using the same protocol. Moreover, it is noteworthy that as Raspberry Pi is the client and Node.js server will be transmitting

values, the Pi is required to initiate the connection with the server. Furthermore, by using a single communication gateway between Raspberry Pi and Node.js server, the architecture ensures that only one channel exists between the two entities.

WebSocket is a bidirectional, full-duplex communication protocol between a client and server over persistent, single-socket connection [19]. Being event-driven and providing low latency, it enables real-time communication for delightful user experiences. WebSockets are a perfect fit for applications such as Remote Labs as it can handle server-sent events and provides more flexibility than HTTP when building applications requiring frequent data exchanges [19].

B. Implementation of the Proposed 3-Layer Software Architecture using WebSockets

To implement the vital WebSocket communication in the proposed implementation, Socket.IO has been used. Socket.IO is a library built on top of WebSocket protocol that provides additional guarantees like HTTP long-polling and automatic reconnection [20]. Using Socket.IO, the clients and server can directly communicate by 'emitting' messages to each other.

The 3-layer architecture proposed here is comprised of -

- 1) *Frontend* - The user interface is required for the clients to conduct experiments, provide inputs and display outputs interactively. In our implementation, front end directly sends the selected circuit operation mode to backend server on receiving them from the user using Axios.
- 2) *Backend Server* - A Node.js-based backend server is deployed to communicate with the hardware node directly. The server sits as the connecting bridge between user and experiments, making it a crucial component to establishing the communication pipeline. It is responsible for getting inputs from user and transferring them to Raspberry Pi. Moreover, using Socket.IO's Server API, it creates a WebSocket server listening to the Raspberry Pi client to establish communication.
- 3) *Hardware Node* - This component comprises KVL experiment. The Raspberry Pi, using the Python library for Socket.IO, becomes a client and connects to deployed Node.js server, ready to receive inputs and send outputs.

Once the experiment is live and connected to the internet, Raspberry Pi becomes a WebSocket client and automatically tries to establish a connection to Node.js server using its predetermined IP address. Once connected, Step I (Fig. 8) of experimentation involves the frontend communicating inputs to backend server using Axios calls. Step II includes the server emitting received input to Raspberry Pi, which computes the output and emits it back to server directly as Step III, without any middleware. In Step IV, server resolves the promise sent earlier and sends output to the frontend to be displayed.

V. RESULTS

The implementation of the 4-layer architecture using Blynk and proposed 3-layer architecture using WebSockets have been compared on parameters like observed latency and round trip

time (RTT) to test their performance and efficiency when multiple users access the KVL experiment simultaneously.

Latency is the total time taken for a data packet to travel from source to destination in a network and is an important metric for interactive real-time applications. In the KVL experiment, latency has been computed twice in the communication pipeline. First, when input is sent from server to Raspberry Pi and second, when output is sent from hardware to Node.js server. These latencies include the Blynk Cloud when measured for the existing 4-layer architecture implementation. The time taken for output computation is also measured. On the other hand, (RTT) refers to the total time taken to send input from the server, compute output and receive output back at the server. To compare the RTT, the time interval is taken between giving the input and receiving the output in epoch time to the nearest millisecond. While measuring for multiple access, measurements have been taken for the time between the first input and the last output.

Note that the architectures differ in server-hardware communication while frontend-backend communication is same. Moreover, time taken to display values on the interface is highly dependent on user device performance, browser versions, cache, network connection speed etc. Hence, it is reasonable to compare previous 4-layer and proposed 3-layer architecture in terms of latency and RTT between server and hardware nodes only, considering the 1-second delay to prevent interruption of thought flow. All tests have been conducted with the hardware connected to 4G Mobile Hotspot configured to 2.4GHz.

A. Individual Latencies and RTT

Table II shows the values obtained for latency in a single user access to hardware node. It also provides a breakdown of the time taken by individual components in the communication pipeline. As is evident from the table, latency observed for a single user in 4-layer implementation is around 852 ms, whereas it is just 48 ms in the proposed implementation using WebSockets. The slightly higher latency from Raspberry Pi to server can be attributed to synchronous nature of implementation of python client in Raspberry Pi and slower read/write from Raspberry Pi memory when compared to Node.js server.

Moreover, considering that human thoughts are disrupted after a delay of 1 second in interactive systems, proposed WebSocket implementation fulfils this limit with seemingly instantaneous results during single access, which is in contrast to when Blynk is used as the delay is fairly perceivable even when output generation time is similar for both.

When multiple users perform the experiment, inputs are queued and provided to hardware sequentially. This increases the latency for users who are queued towards the end, as they accessed the experiment later. Fig. 9 shows that when a different number of users attempt simultaneous experimentation, the RTT rises linearly due to queueing but WebSockets exhibit significantly lower delay for the last user in the queue. In 4-layer architecture, the user at the 100th position receives their output after approximately 85 seconds, whereas with WebSockets, it is received only 5.5-5.7 seconds later.

TABLE II: Latency values in communication between backend server and hardware node with existing 4-layer and proposed 3-layer architecture using Blynk, WebSockets respectively.

Latency in Communication	Blynk	WebSockets
Input from Node.js server to Raspberry Pi (in ms)	407	5.5
Output Computation at Hardware Node(in ms)	36	37
Output from Raspberry Pi to Node.js server(in ms)	442	6.5
RTT for single hit (in ms)	852	48

TABLE III: Individual latencies as % of RTT.

Individual Latency	Blynk	WebSockets
Input from Node.js server to Raspberry Pi (in ms)	46	11.4
Output Computation at Hardware Node(in ms)	4.1	75.1
Output from Raspberry Pi to Node.js server(in ms)	49.9	13.5

B. Number of Users

As is apparent, by using WebSockets one can serve much more people in the same time duration when compared to 4-layer architecture implementation using Blynk. The number of users who can concurrently experiment on the same hardware is directly dependent on RTT and latency in the communication pipeline while ensuring that values obtained are correct and accurate. Existing architecture can hence accommodate $1/0.852 \approx 1$ user, whereas WebSockets can accommodate $1/0.048 \approx 20$ users per second, which is clearly an advancement over standard implementations.

C. Further improvements for 3-layer architecture

It can be observed Table II, the 3-layer architecture using WebSockets has a faster response time. Moreover, it is evident from Table III that just 4.1% of the RTT is occupied with output calculation, and a staggering 95.9% is just transferring data packets when using Blynk. However, when we use WebSockets, this share of transmitting data packets gets reduced to 25% only, highlighting that the latency is much less in the 3-layer architecture. Observe that while communicating the inputs is the most time-consuming factor in the 4-layer architecture, output computation takes 75.1% of the total RTT, becoming the bottleneck for WebSocket implementations.

By using solid-state relays, time taken in switching between various configurations can be reduced to improve the response time. Solid-state relays with an operate and release time of 1 ms each [21], decreasing the 15 ms switching time in electromagnetic relays to 2 ms and reducing the latency further to around 33-35 ms at times of multiple access, providing outputs to around 30 users in less than a second. As output generation does not consume a large share of the latency in 4-layer implementations, there won't be an increase in scalability even after improving the hardware.

Moreover, if the queue implementation is shifted to the Raspberry Pi and the Python implementation is made asynchronous as well, we can eliminate the hardware's need to wait for inputs after processing an output. This would lead to parallel processing and increased throughput as while output is being sent, another computation is in progress. This maintains our bottleneck to be the 24 ms computation time at the

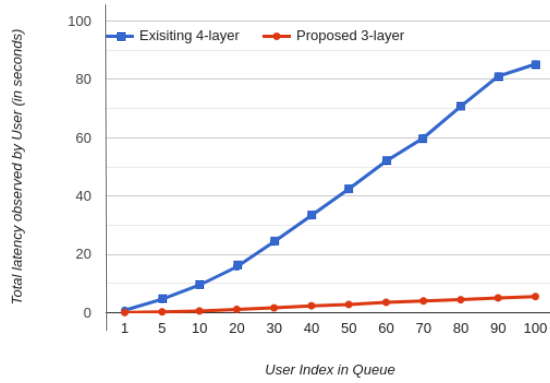


Fig. 9: Total latency observed depending on the number of users accessing simultaneously.

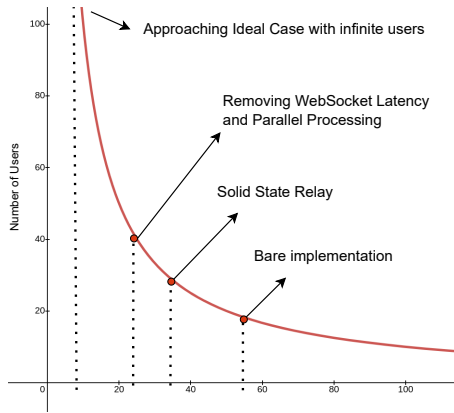


Fig. 10: Number of users vs latency of system.

hardware. Also, WebSockets are known to add virtually no latency and with latency now dependent only on the network used; by using better networks to run the tests, one can essentially remove the communication latency, leaving only the 24 ms and increasing the number of users to $1/0.024 \approx 41$. The number of users which can obtain their results in under one second is hence inversely proportional to latency in the system and can be represented as Fig. 10.

Authentication and authorization of data can be implemented to enhance security of such systems. Moreover, WebSocket implementations are prone to Man in the Middle attacks making it important to use Secure Websockets as they add SSL/TLS encryption for secure communication.

VI. CONCLUSION

This paper highlights the possibility of scaling Remote Labs for experiments like KVL which can provide instantaneous outputs by allowing multiple users to simultaneously access hardware setups without affecting the experience for other users. This work has emphasized on how 4-layered architecture used for web-based Remote Lab implementations is inefficient to be scaled using multi-user multiplexing and cannot provide concurrent access to multiple users. A 3-layer architecture leveraging WebSockets has been proposed and compared with existing 4-layer implementations using Blynk as the middleware. The results suggest that latency experienced

by users is much less while using WebSockets, and 3-layer architecture can provide accurate outputs to 20 users with bare implementation and up to 41 users by using the solid-state drive and parallel processing. Hence, by over-viewing the results obtained, it can be concluded that the 4-layered architecture implemented using Blynk, is inadequate for efficiently scaling Remote Labs for experiments with instantaneous outputs and that the proposed 3-layer architecture using WebSockets for communication between server and hardware node can allow concurrent user access and implement multi-user multiplexing to scale Remote Labs.

VII. ACKNOWLEDGEMENT

This work was funded partially by Enabler Grants made available through Kohli Center for Intelligent Systems (KCIS), with no conflicts of interest. The authors also acknowledge the receipt of a grant from TIH Foundation for IoT & IoE at IIT Bombay under the TIH-IoT CHANAKYA Fellowship Program 2022-23 (3) (Sanction Letter No. TIH-IoT/2023-03/HRD/CHANAKYA/SL/CFP-016)

REFERENCES

- [1] Lea, P., *Internet of Things for Architects*, Packt Publishing, 2018.
- [2] F. Valencia de Almeida et al., "Teaching Digital Electronics during the COVID-19 Pandemic via a Remote Lab," *Sensors*, vol. 22, no. 18, p. 6944, Sep. 2022, doi: 10.3390/s22186944.
- [3] Bangert K et al., "Remote practicals in the time of coronavirus, a multidisciplinary approach.," *Int. Journal of Mechanical Engineering Education*, vol. 50, no. 2, pp. 219–239, 2022.
- [4] Borish et al., "Undergraduate student experiences in remote lab courses during the COVID-19 pandemic," *Phys. Rev. Phys. Educ. Res.*, vol. 18, pp. 020105, Jul 2022.
- [5] S. Kurukunda, C. Trigona, S. Baglio, "Laboratory Activity during COVID- 19 as a "Virtual Experience": Restriction or Chance?," in *17th Int. Multi-Conference on Systems, Signals Devices (SSD)*, 2020, pp. 349–353.
- [6] "UNILabs," <https://unilabs.dia.uned.es/>.
- [7] "MIT iLabs," <http://icampus.mit.edu/projects/ilabs/>.
- [8] K. S. Viswanadh et al., "CV and IoT-based Remote Triggered Labs: Use Case of Conservation of Mechanical Energy," in *9th Int. Conference on Future Internet of Things and Cloud (FiCloud)*, 2022, pp. 100–106.
- [9] A. Das et al., "Using Miniature Setups and Partial Streams for Scalable Remote Labs (Accepted)," in *10th Int. Conference on Future Internet of Things and Cloud (FiCloud)*, Marrakesh, Morocco, 14-16 August 2023.
- [10] "European Go-Labs Initiative," https://premium.golabz.eu/sites/premium/files/inline-files/Go-Lab_D4.5.pdf.
- [11] "LabsLand Network," <https://labsland.com/en>.
- [12] Najib ABEKIRI et al., "Platform for hands-on remote labs based on the ESP32 and NOD-red," *Scientific African*, vol. 19, pp. e01502, 2023.
- [13] Villar-Martínez, Aitor et al., "Improving the Scalability and Replicability of Embedded Systems Remote Laboratories Through a Cost-Effective Architecture," *IEEE Access*, vol. 7, pp. 164164–164185, 2019.
- [14] Hayt, William Hart, and Kemmerly, Jack E. , *Engineering circuit analysis / William H. Hayt, Jr., Jack E. Kemmerly* , McGraw-Hill New York, 5th ed. edition, 1993.
- [15] "Axios - Promise Based HTTP Client," <https://axios-http.com/>.
- [16] "Blynk APIs," <https://docs.blynk.io/en/blynk.cloud/https-api-overview>.
- [17] J. Nielsen, " Usability Engineering. Amsterdam, The Netherlands: Elsevier, 1994.," <https://dl.acm.org/doi/pdf/10.5555/2821575>.
- [18] "Blynk Protocol," <https://docs.blynk.io/en/blynk-library-firmware-api/blynk-protocol>.
- [19] A. Lombardi, *WebSocket: Lightweight Client-Server Communications*, O'Reilly Media, 2015.
- [20] "Socket.IO Library built on top of WebSockets," <https://socket.io>.
- [21] "G3MB-202P Datasheet (PDF) - Omron Electronics LLC," <https://pdf1.alldatasheet.com/datasheet-pdf/view/333500/OMRON/G3MB-202P.html>.